

Entrepreneurship Coach Documentation

By: Sai charan B k

Index:

1. Architecture
2. Frontend Team
3. Backend
4. Async and Worker Team
5. QA Team (ReAssigned at End)
6. Github Rules
7. Handoffs
8. anti-Vibe

Deadlines:

Frontend and Backend (TIPSC and DFV API's):- Jul 6, 2026 & Jul 7, 2026

Async Team :- Jul 8, 2026

Fully working by Jul 14, 2026

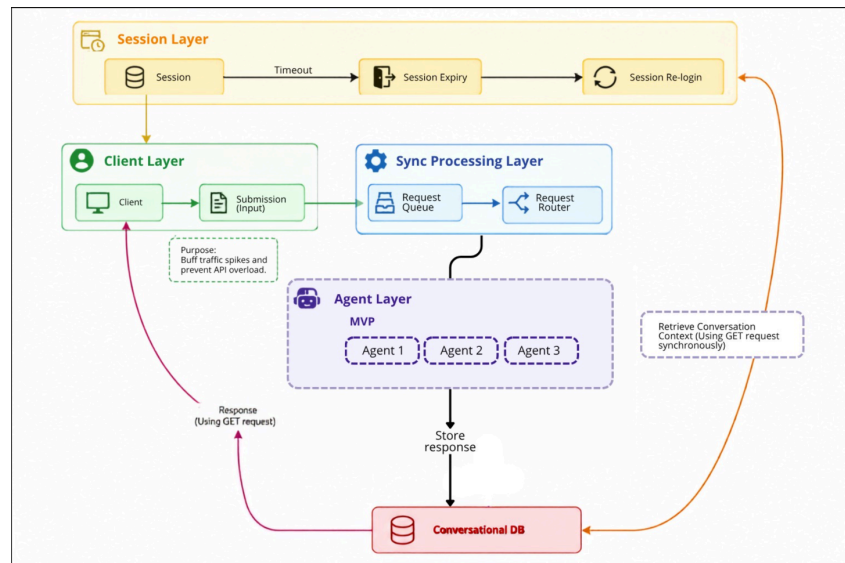
QA testing start by Jul 15, 2026 to Jul 20, 2026

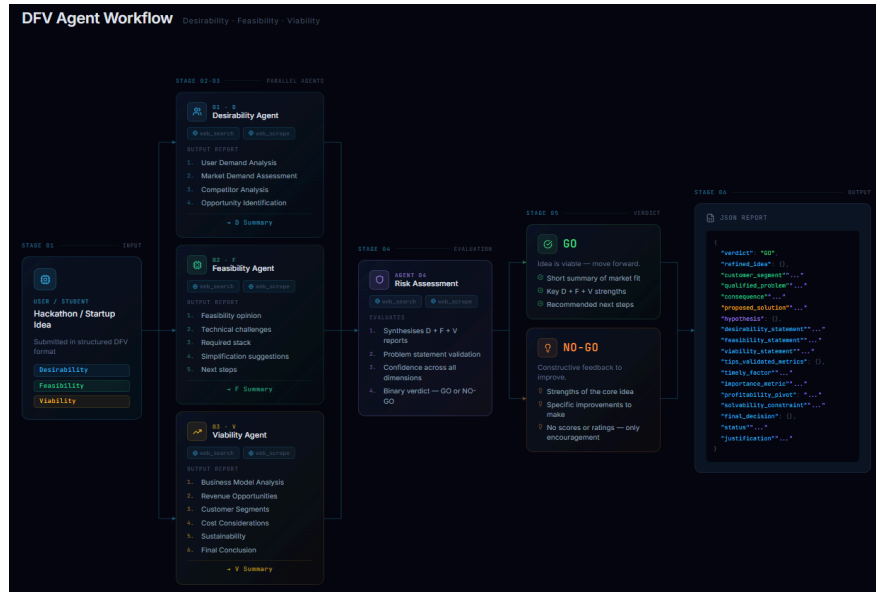
Poster:- Jul 14, 2026

Architecture Team (ALL Members)

- Define the exact user journey for student and mentor.
 - The student answers the pre-eval questions after which there is a validation check followed by regulatory mapping. The context of these is passed to ethics gate which checks through on legal, ethical, etc bases and exits if it violates any.
 - From the ethical phase a compliance context is built and that is passed to the tipsc scoring and followup agent only if the compliance flag is true.
 - Now, the tipsc scores based on the pre-eval answers with validation and compliance context. Then if there is a need for followup then there are max 3 followups asked and tipsc are scored and gives the reason for the same after each followup and finally gives results consisting of tipsc score, whether its ready for dfv and any compliance issue.
 - The student inputs the dfv based on the results of tipsc only when it is passed by tipsc, where the D, F and V agents run parallelly for Desirability, Feasibility, and Viability reports which are passed to the evaluation agent which analysis the report and gives GO or NO-GO decision with a summary and recommendations along with a JSON report .

- Define userSession
 - Each user has a single session, if they want to discard the idea and start a new one, they can save it as an md and restart the session. Score gets added to DB which is already there but gets changed every time they restart the session and reach the end of each phase) MongoDB schema.
- Define per-flow input and output shapes.(Ignore)
- Define Kafka topic names and message payloads.
 - Preeval
 - Followup
 -
- Define RBAC: student, mentor
 - User/Team Level. One user can edit and modify only their own workspace and can ONLY VIEW and not edit other users' workspaces.Each user has exactly ONE session (with refresh options)
 - Mentor will have only a view access to the users workspace but there will be a Mentor Chatbox implemented for Mentor to add any comments if any and inform the team of any changes or pivots.
- Define system state machine (all allowed transitions).





- Define task and acceptance criteria for all teams(AGIS). (IGNORE FOR NOW)
- Define which Local models to be used (or) Cloud Models to be used (Cloud can change later)
 - Mistral 7B
 - Qwen 3.5 9B
- Create sequence diagrams for PS submission, queueing, worker completion, and UI fetch.
- Output: [see docs/* in repo structure](#)

Definition of done

- Each story has testable acceptance criteria.
- Each state transition has an allowed source and destination.
- Each event has versioned payload fields.
- No one starts implementation without these documents approved.

Frontend team

Goal: deliver a production-grade React UI that explains the flow and lets students and mentors operate clearly.

Tasks

- Build Page 1 platform intro page: what the platform does, what each agent does, what each flow does, what students should expect, student vs mentor explanation.
- Build authenticated student workspace with one PS(problem Statement) Session dashboard.
 - Pesu-auth from pes-dev api using SRN and password for each user and then assigned to their team sessions.
- Add sections in page 2 for Idea Evaluation, DFV, and Customer Discovery Planner.
 - Make a tabular section which will contain all 3 sections TIPSC, DFV and JTBD out of which only TIPSC will be accessible to the student at the initial stage and further DFV will only be unlocked only when the idea is verified by TIPSC and then JTBD when idea is DFV verified.
- Build queue states: queued, processing, done, failed, retry available.
- Build design system: tokens, typography, button, card, form, badge, loading, error, empty state components.
- Build output rendering components for each flow.
- Build submission form for PS + TIPSC inputs. Wire to mock API client. Show queued status after submit
- Implement polling for session status updates.
- Build a mentor dashboard to list teams, filter by status, and inspect outputs. (ignore for now)
- Build async status poller: fetch `/userSession/{id}` every 5s, update status badge, stop on done or failed.
- Build DFV submission form - desirability, feasibility, viability, student inputs. Trigger Flow 2 on submit
- Build mentor dashboard - list all PS by team, filter by flow status, click-through to detail
- Build PS timeline / audit view. Show all events for a session in chronological order
- Build Customer Discovery Planner section — trigger Flow 3, show JTBD elements and interview plan output
- Add audit timeline UI for PS events.
- Add strong form validation and empty/error/loading states.
- Polish - loading skeletons, empty states, error banners, mobile layout pass, accessibility fixes

Required output format

- Screen inventory.

- Component map.
- Props contract sheet.
- Route map.
- Final UI with responsive behavior notes.

Definition of done

- Every screen maps to an approved story.
- No hardcoded mock shape that diverges from backend schema.
- Every async view has loading, success, empty, and error states.
- Keyboard and mobile usability pass manual checks.

Backend Platform team

Goal: deliver secure and clean APIs plus MongoDB persistence.

Tasks

- Set up FastAPI project in `backend/`, project structure, env config, MongoDB connection.
- Implement auth and user/team/mentor authorization.
- Write Pydantic models for `userSession`, all flow inputs and outputs, user, team
- Build POST endpoint to create Team Session.
- Implement `POST /userSession` — validate input, write `userSession` to MongoDB, return `{userSession_id, status: queued}`.
- Implement `GET /userSession/{id}` — return full session with flow statuses and outputs.
- Build GET endpoint to fetch Team Session and outputs.
- Implement `POST /userSession/{id}/trigger/{flow}` — validate PS state, publish correct Kafka message, update status
- Implement audit log endpoint `GET /userSession/{id}/history`. Write audit events on every state change.
- Build endpoints to trigger each flow asynchronously.
- Implement MongoDB models and indexes.
- Add idempotency handling to avoid duplicate submissions.
- Add status update services for worker completion.
- Add audit log writes for every important event.
- Add validation, error codes, and API versioning.
- Build mentor-scoped API — `GET /mentor/userSession` with filters. Enforce RBAC.
- Write mentor comment API. Add `POST /userSession/{id}/comments`

Required output format

- API spec in OpenAPI. (Refer this [Link](#))
- Mongo collection design.
- Error catalog.

Definition of done

- Every endpoint has request/response examples.
- Input validation rejects malformed payloads.
- DB writes are traceable with timestamps and actor.
- Duplicate trigger calls do not create corrupt state.

Async and Worker Systems team

Goal: make the queue-backed flow execution reliable and observable.

Tasks

- Set up Kafka topic strategy for `idea_eval`, `dfv`, `discovery`, `notifications`.
- Set up Kafka producer inside FastAPI using `aiokafka`. Write topic config and base producer class.
- Implement consumer workers for each flow.
- Set up consumer worker skeleton for all three flows.
- Write base consumer class with retry and dead-letter logic and poison message strategy.
- Write worker state updates back to MongoDB.
- Implement producer call inside `POST /userSession` after DB write. Publish to `userSession.idea_eval` topic.
- Implement Idea Eval consumer - consume message, update status to `running`, call CrewAI TIPSC flow, write output back to MongoDB, update status to `done`, publish to `userSession.notifications`
- Implement DFV consumer - consume from `userSession.dfv`, call CrewAI DFV flow with student-provided DFV context, write output, update status.
- Implement Discovery consumer - consume from `userSession.discovery`, call CrewAI Customer Discovery Planner flow, extract JTBD elements and interview plan, write output, update status
- Implement worker logs with correlation IDs.
- Emit notification events on completion/failure.
- Add timeout handling and job recovery logic.
- Add notification worker — consume `userSession.notifications`, push UI-visible event (polling-ready signal) to MongoDB.
- Build a lightweight job monitor dashboard or admin endpoint.

Required output format

- Topic map.
- Message schema spec.
- Retry and failure matrix.
- Worker runbook.

Definition of done

- A submitted session can be traced from API call to Kafka event to DB output.
- Retries do not duplicate final output.
- Failed jobs are visible and actionable.
- Each message carries `userSession_id` and `correlation_id`.

QA Team

Goal: enforce discipline, release confidence, and environment stability.

Tasks

- Define test pyramid: unit, API integration, queue integration, end-to-end.
 - Create CI pipeline for lint, typecheck, unit tests, contract tests, and build.
 - Set up dev, staging, and prod configs.
 - Add seeded test data for student and mentor roles.
 - Define release checklist and rollback procedure.
 - Load test async submission path for concurrent users.
 - Verify observability: logs, metrics, alerts.
 - Gate merges on checklist compliance.
 - Write and run manual test cases for all 3 flow paths (happy path, failure, retry, mentor view).
 - Run integration tests, document bugs, assign to correct team for fix.
 - bugs, flaky flows, edge cases, blocked dependencies.
- This rhythm helps the team think deeply, detect weak assumptions early, and avoid “looks finished but breaks in reality” delivery.

Required output format

- Test strategy.
- CI/CD workflow document.
- Release checklist.
- Environment variable matrix.

Definition of done

- Core flow passes in staging end to end.
- Failure paths are tested, not assumed.
- Build is reproducible from repository alone.
- Monitoring shows queue lag, worker failures, and API errors.

Github Rules

Repository Architecture

Goal: Establish a clean root structure to ensure clear ownership and streamlined reviews.

AGIS-2026-Experiments/

```
|— docs/
| |— schema.json
| |— api-spec.md
| |— flow-shapes.md (optional)
| |— rbac.md (optional)
| |— user-journey.md
| |— model-config.md
| |— sequence-diagrams.md(optional)
| |— frontend-arch.md
| |— backend-arch.md
|— frontend/
|— backend/
| |— routes/
| |— models/
| |— kafka/
| |— workers/
| |— main.py
|— DFV-agent/
|— TIPSC-Agent/
|— customer-interview-planner-agent/
```

Structural Guidelines

- The `docs/` directory is the final authority for all system architecture. No implementation begins without approved documents in this directory.
- The `frontend/` module is restricted to presentation logic only. No API calls outside the service layer, no business logic, no hardcoded data shapes.
- API route definitions live in `backend/routes/`, data schemas in `backend/models/`, async consumers in `backend/workers/`, and Kafka configuration in `backend/kafka/`.
- Agent modules (`DFV-agent/`, `TIPSC-Agent/`, `customer-interview-planner-agent/`) must remain fully encapsulated. They are invoked exclusively by worker processes. No other module may import or call them directly.

Branching Strategy

Branch Classifications

Branch	Function	Merge Authority
<code>main</code>	Production-grade releases only	Owner only, 2 approvals required
<code>develop</code>	Daily integration cycle	Team via peer review PR
<code>develop/frontend</code>	Frontend team integration	Frontend peer, then PR to develop
<code>develop/backend</code>	Backend team integration	Backend peer, then PR to develop

develop/workers	Workers team integration	Workers peer, then PR to develop
feature/*, fix/*	Discrete task execution	PR into team sub-branch only
docs/*	Architecture document changes	PR into develop, owner approval

Naming Conventions:

Team sub-branches

- develop/frontend
- develop/backend
- develop/workers
-

Feature branches

- feature/frontend/F-01-intro-page
- feature/frontend/F-03-ps-tipsc-form
- feature/frontend/F-08-poller-hook
-
- feature/backend/B-01-create-session
- feature/backend/B-04-pydantic-models
- feature/backend/B-06-audit-log
-
- feature/workers/W-01-idea-eval-consumer
- feature/workers/W-03-dfv-consumer
- feature/workers/W-05-notification-worker

Fix branches

- fix/frontend/F-03-validation-bug
- fix/backend/B-01-duplicate-session
- fix/workers/W-01-retry-deadlock

Docs branches

- docs/A-02-schema-definition
- docs/A-04-kafka-contracts

Workstream Sequence:

- feature/backend/B-01 → PR → develop/backend
- feature/frontend/F-03 → PR → develop/frontend
- feature/workers/W-01 → PR → develop/workers

- develop/backend → PR → develop (sub-branch lead promotes)
- develop/frontend → PR → develop
- develop/workers → PR → develop

- develop → PR → main (owner final authorization)

- All development branches originate from their respective team sub-branch.
- Feature and fix PRs target the team sub-branch only, never `develop` or `main` directly.
- Sub-branch leads promote stable team work into `develop` after peer verification.
- `develop` merges into `main` only after full integration verification and owner authorization.

PR & Peer Review Protocol

Non-Negotiable Rules

- Direct commits to `main`, `develop`, and `develop/*` branches are strictly prohibited.
- Every pull request must link a Task ID in the title and description.
- PR descriptions must include the specific acceptance criteria from the task card.
- Scope is limited to one logical unit of implementation per PR.
- WIP code is never merged. A task is either acceptance-criteria-complete or it stays in the branch.
- AI-assisted code requires a line-by-line developer walkthrough before merge. The author must be able to explain every line without referencing the prompt.

Standard PR Checklist

- Inventory of all modified files with a one-line description of what changed in each.
- Plain-English explanation of logic — trigger, process, output.
- Step-by-step local validation procedure so the reviewer can reproduce the result.
- **Frontend:** screenshots of all 4 states — loading, success, empty, error.
- **Backend:** representative request and response payload examples.
- **Workers:** proof of idempotency and documented error handling behavior.

Approval Matrix

PR path	Required approvals
<code>feature/*</code> → <code>develop/frontend</code>	1 frontend peer
<code>feature/*</code> → <code>develop/backend</code>	1 backend peer
<code>feature/*</code> → <code>develop/workers</code>	1 workers peer
Worker PR touching DB schema	1 workers peer + 1 backend lead
<code>develop/*</code> → <code>develop</code>	Sub-branch lead + QA sign-off
<code>docs/*</code> → <code>develop</code>	Owner approval
<code>develop</code> → <code>main</code>	2 lead approvals + owner final authorization

Testing

Active Pipelines:

- `frontend-ci.yml`: Dependency audit, linting, type safety, and build verification.
- `backend-ci.yml`: Pytest suite execution and OpenAPI schema validation.
- `workers-ci.yml`: Worker logic validation and Kafka message-schema checks.
- `docs-check.yml`: Enforces architectural documentation presence before merging code.

(Can be implemented if needed)

Minimum Release Criteria:

- Frontend: Successful production build.
- Backend: Full test coverage and schema compliance.
- Workers: Verified idempotency and retry mechanics.
- System Integration: Mandatory E2E(End-To-End) checklist sign-off by QA.

Handoffs

Submission path

- Student can create a team session.
- Session is saved to MongoDB with correct schema.
- Kafka message is published on submission.
- Frontend shows queued status immediately.

Idea Evaluation flow

- Worker consumes message and updates status to running.
- CrewAI TIPSC flow executes and produces output.
- Output is written to MongoDB correctly.
- Status updates to done.
- Frontend renders output.

DFV flow

- Student can submit DFV inputs after idea eval is done.
- Worker executes DFV flow and writes output.
- Frontend renders DFV coach feedback and go/no-go.

Customer Discovery flow

- Student can trigger discovery planner.
- Worker executes discovery flow and writes JTBD elements and interview plan.
- Frontend renders structured interview plan.

Mentor dashboard (could have)

- Mentor can see all teams and team sessions.
- Mentor can view full session detail.
- Mentor can leave comments.
- Audit trail shows all events.

Reliability

- Failed flow updates status to failed with error context.
- Worker retries do not duplicate DB writes.
- Dead-letter queue catches unprocessable messages.
- Duplicate trigger calls are blocked by state check.

Anti-Vibe-Coding Rules (Non-Negotiable)

- No PR merged if Testing fails
- No endpoint merged without Postman example (For API).
- No UI component merged without all 4 states: loading, success, empty, error.
- No worker merged without an idempotency test.
- AI-generated code requires line-by-line review and local execution before merge.
- A Person should know each and every module and its data flow which is build along with the team

Assignment of Teams

Teams	No. of People	Team Members
Frontend	3	Abhay, Aditi, Urav
Backend	5	Palash, Bhavesh, Sujal, Vijay, Parthiv
Workers / Async	3	Kishan, Ashwin, Krrish
QA / Fixers (reassigned at end)	7, 4	